SQL LAB: STUDENT DATABASE

Learn SQL Commands Through One Practical Dataset

A hands-on lab guide covering DDL, DML, DQL, DCL, TCL, constraints, functions, subqueries, views and indexes.

Learning Outcomes

By the end of the lab, students should be able to:

- Create a database and design a Student table
- Insert, retrieve, update and delete records
- Use conditions, sorting, grouping and aggregate functions
- Modify table structures and apply constraints
- Work with transactions, permissions, views and indexes
- Solve practical database tasks using SQL commands

SQL Command Categories

We will practise each category using the same Student database.

Category	Full form	Main commands
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	INSERT, UPDATE, DELETE
DQL	Data Query Language	SELECT
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT

Step 1 — Create the Database

DDL commands: CREATE and USE

```
CREATE DATABASE college_db;

USE college_db;
```

- CREATE DATABASE creates a database.
- USE selects it for subsequent commands.
- Run these commands before creating the table.

Step 2 — Student Table Design

Table name: students

Column	Data type	Constraint / default	Purpose
student_id	INT	PRIMARY KEY	Unique student ID
student_name	VARCHAR(100)	NOT NULL	Student name
email	VARCHAR(100)	UNIQUE	Email address
gender	VARCHAR(10)	—	Gender
age	INT	—	Student age
department	VARCHAR(50)	—	Department
semester	INT	—	Current semester
cgpa	DECIMAL(3,2)	—	CGPA
admission_date	DATE	—	Admission date
city	VARCHAR(50)	DEFAULT 'Dhaka'	City
is_active	BOOLEAN	DEFAULT TRUE	Active status

Step 3 — CREATE TABLE

Define columns, data types and constraints.

```
CREATE TABLE students (
  student_id INT PRIMARY KEY,
  student_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE,
  gender VARCHAR(10),
  age INT,
  department VARCHAR(50),
  semester INT,
  cgpa DECIMAL(3,2),
  admission_date DATE,
  city VARCHAR(50) DEFAULT 'Dhaka',
  is_active BOOLEAN DEFAULT TRUE
);
```

- PRIMARY KEY identifies each row.
- NOT NULL requires a value.
- UNIQUE prevents duplicate emails.
- DEFAULT supplies a value when omitted.

Step 4 — INSERT Sample Records

Insert multiple rows in one statement.

```
INSERT INTO students
(student_id, student_name, email, gender, age, department,
 semester, cgpa, admission_date, city, is_active)
VALUES
(101,'Rahim Ahmed','rahim@gmail.com','Male',22,'CSE',6,3.75,'2023-01-15','Dhaka',TRUE),
(102,'Karim Hasan','karim@gmail.com','Male',23,'EEE',8,3.50,'2022-01-10','Chittagong',TRUE),
(103,'Nusrat Jahan','nusrat@gmail.com','Female',21,'CSE',4,3.90,'2024-01-12','Dhaka',TRUE),
(104,'Sadia Akter','sadia@gmail.com','Female',22,'BBA',6,3.60,'2023-01-18','Sylhet',TRUE),
(105,'Tanvir Hossain','tanvir@gmail.com','Male',24,'CSE',8,3.25,'2022-01-20','Rajshahi',FALSE);
```

Add More Records and Verify

Continue adding records, then inspect the table.

```
INSERT INTO students
(student_id, student_name, email, gender,
 age, department, semester, cgpa,
 admission_date, city, is_active)
VALUES
(106,'Mim Akter','mim@gmail.com','Female',21,'EEE',4,3.80,'2024-
01-15','Dhaka',TRUE),
(107,'Sakib
Islam','sakib@gmail.com','Male',23,'BBA',7,3.40,'2022-07-
10','Khulna',TRUE),
(108,'Farzana
Rahman','farzana@gmail.com','Female',22,'CSE',6,3.95,'2023-01-
25','Dhaka',TRUE);
```

```
SELECT * FROM students;
```

Count all records

```
SELECT COUNT(*) FROM students;
```

DQL — SELECT and Filtering

Retrieve only the records you need.

Topic	SQL command
All columns	SELECT * FROM students;
Selected columns	SELECT student_name, cgpa FROM students;
WHERE	SELECT * FROM students WHERE cgpa > 3.50;
AND	SELECT * FROM students WHERE department='CSE' AND cgpa > 3.50;
OR	SELECT * FROM students WHERE department='CSE' OR department='EEE';
NOT	SELECT * FROM students WHERE NOT department='CSE';
DISTINCT	SELECT DISTINCT department FROM students;
IS NULL	SELECT * FROM students WHERE email IS NULL;

Filtering, Sorting and Limiting

Operators help narrow down query results.

Topic	SQL command
ORDER BY	SELECT * FROM students ORDER BY cgpa DESC;
LIMIT	SELECT * FROM students LIMIT 5;
BETWEEN	SELECT * FROM students WHERE age BETWEEN 21 AND 23;
IN	SELECT * FROM students WHERE department IN ('CSE','EEE');
LIKE — starts with R	SELECT * FROM students WHERE student_name LIKE 'R%';
LIKE — contains 'an'	SELECT * FROM students WHERE student_name LIKE '%an%';
Comparison	SELECT * FROM students WHERE semester >= 6;

DML — UPDATE and DELETE

Change or remove selected records.

UPDATE a student's CGPA

```
UPDATE students
SET cgpa = 3.95
WHERE student_id = 101;
```

DELETE a selected record

```
DELETE FROM students
WHERE student_id = 108;
```

SAFETY RULE: Always check the WHERE condition before UPDATE or DELETE. Without WHERE, every row may be affected.

```
SELECT * FROM students WHERE student_id = 101;
```

Aggregate Functions

Summarize values across multiple rows.

Function	SQL command
COUNT	SELECT COUNT(*) FROM students;
AVG	SELECT AVG(cgpa) FROM students;
MAX	SELECT MAX(cgpa) FROM students;
MIN	SELECT MIN(cgpa) FROM students;
SUM	SELECT SUM(age) FROM students;

```
SELECT
  COUNT(*) AS total_students,
  AVG(cgpa) AS average_cgpa,
  MAX(cgpa) AS highest_cgpa
FROM students;
```

Aggregate functions return summary values.

GROUP BY and HAVING

Group records and filter the groups.

```
-- Count students in each department
SELECT department, COUNT(*) AS total
FROM students
GROUP BY department;
```

```
-- Departments with average CGPA above 3.50
SELECT department, AVG(cgpa) AS avg_cgpa
FROM students
GROUP BY department
HAVING AVG(cgpa) > 3.50;
```

- GROUP BY creates groups with the same value.
- HAVING filters grouped results.
- WHERE filters individual rows before grouping.

DDL — ALTER and RENAME

Change the structure or name of a table.

Task	MySQL command
Add a column	ALTER TABLE students ADD phone VARCHAR(15);
Modify a column	ALTER TABLE students MODIFY student_name VARCHAR(150);
Rename a column	ALTER TABLE students RENAME COLUMN city TO address;
Drop a column	ALTER TABLE students DROP COLUMN phone;
Rename table	RENAME TABLE students TO student_info;
Rename back	RENAME TABLE student_info TO students;

Note: These examples use MySQL syntax.

DDL — TRUNCATE and DROP

Use destructive commands only in a disposable practice database.

```
TRUNCATE TABLE students;
```

```
DROP TABLE students;
```

TRUNCATE

- Removes all rows from the table.
- Keeps the table structure.

DROP

- Removes the table itself.
- Deletes both data and structure.

Do not run these commands on important or production data.

Constraints — Rules for Reliable Data

Use a separate practice table to demonstrate these constraints.

```
CREATE TABLE student_demo (
  student_id INT PRIMARY KEY,
  student_name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE,
  cgpa DECIMAL(3,2),
  city VARCHAR(50) DEFAULT 'Dhaka',
  CONSTRAINT chk_cgpa CHECK (cgpa BETWEEN 0 AND 4.00)
);
```

- PRIMARY KEY: unique row identifier.
- NOT NULL: value is required.
- UNIQUE: no duplicate values.
- DEFAULT: supplies a fallback value.
- CHECK: validates a condition.

Subqueries, Views and Indexes

Useful techniques for querying and organizing data.

Subquery

```
SELECT * FROM students
WHERE cgpa > (
 SELECT AVG(cgpa)
 FROM students
);
```

View

```
CREATE VIEW top_students AS
SELECT student_id, student_name, cgpa
FROM students
WHERE cgpa >= 3.75;
```

Index

```
CREATE INDEX idx_department
ON students(department);
```

Read a view: SELECT * FROM top_students;

Drop a view: DROP VIEW top_students;

Drop a MySQL index: DROP INDEX idx_department ON students;

TCL — Transaction Control

A transaction groups related changes into one unit of work.

```
START TRANSACTION;

UPDATE students
SET cgpa = 4.00
WHERE student_id = 101;

SAVEPOINT after_update;

-- Save the changes:
COMMIT;
-- Or undo uncommitted changes:
ROLLBACK;
```

- START TRANSACTION begins a transaction.
- COMMIT saves changes permanently.
- ROLLBACK cancels uncommitted changes.
- SAVEPOINT marks a point inside a transaction.

DCL — GRANT and REVOKE

Manage what a database user is allowed to do.

```
                    -- Create a lab user
CREATE USER 'lab_user'@'localhost'
IDENTIFIED BY 'Lab@12345';

-- Grant selected permissions
GRANT SELECT, INSERT, UPDATE
ON college_db.students
TO 'lab_user'@'localhost';

-- Revoke INSERT permission
REVOKE INSERT
ON college_db.students
FROM 'lab_user'@'localhost';
```

- GRANT gives privileges.
- REVOKE removes privileges.
- Appropriate administrative permissions are required.
- Use secure passwords; do not share accounts.

Student Lab Challenge

Complete these tasks using the students table.

- Create a database named university and a students table.
- Insert at least 10 student records.
- Display all students from the CSE department.
- Find students whose CGPA is greater than 3.50.
- Display students in descending order of CGPA.
- Find the average CGPA of each department.
- Update the city of one student using student_id.
- Delete one selected student record.
- Create a view for students with CGPA above 3.75.
- Count the number of students in each department.

Quick Revision

One database • One table • Many SQL commands

Category	Remember these commands
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	INSERT, UPDATE, DELETE
DQL	SELECT, WHERE, ORDER BY, GROUP BY, HAVING
DCL	GRANT, REVOKE
TCL	COMMIT, ROLLBACK, SAVEPOINT
Additional	Constraints, Aggregate Functions, Subqueries, Views, Indexes

Thank You — Happy Querying!